

RICAN: A Fixed-Rate Binary Representation Framework for Finite Alphabets Using Continued-Fraction Convergents

J. Jesús Martínez Palomo Independent Researcher
Email: enviarcorreoajesusdelaunam@gmail.com

Abstract

This paper introduces RICAN (Representational Isomorphic Coding for Arbitrary Numeration), a mathematical framework for constructing asymptotically tight fixed-rate reversible binary representations for sequences over arbitrary finite alphabets.

The central contribution is a constructive method for selecting block sizes through continued-fraction approximation of $\log_2(B)$. The method identifies convergent-derived block lengths that generate deterministic fixed-rate representations with arbitrarily small absolute rounding gaps along suitable subsequences. Consequently, the normalized per-symbol representation overhead converges to zero, where B denotes the alphabet cardinality.

RICAN does not perform source compression; it minimizes deterministic fixed-rate binary representation overhead.

For a block of length K over an alphabet of size B , the minimum number of bits required by any fixed-length injective binary representation is bounded by

$$n(K) = \lceil K \log_2(B) \rceil$$

bits.

The unavoidable rounding overhead is therefore

$$G_B(K) = \lceil K \log_2(B) \rceil - K \log_2(B).$$

RICAN demonstrates that denominators of continued-fraction convergents of $\log_2(B)$ provide an infinite subsequence of block sizes whose absolute rounding gap approaches zero, and therefore whose normalized per-symbol representation overhead converges to zero.

The framework applies uniformly to the infinite family of finite alphabets

$$\mathcal{A} = \{\Sigma_B : B \geq 2\},$$

where

$$\Sigma_B = \{0, 1, \dots, B-1\}.$$

Power-of-two alphabets achieve exact binary representations with zero overhead. For non-power-of-two alphabets, continued-fraction convergents provide deterministic asymptotically tight block sizes.

For the decimal alphabet ($B = 10$), the practical reference implementation uses the convergent-derived block size

$$K = 643,$$

requiring

$$n(K) = 2136$$

bits and achieving a representation overhead of approximately

$$3.65 \times 10^{-7}$$

bits per symbol.

A larger upper convergent denominator

$$K = 783032$$

corresponds to the convergent

$$\frac{2601184}{783032}$$

in the continued fraction expansion of $\log_2(10)$, satisfying

$$K \log_2(10) = 2601183.99999948 \dots,$$

requiring

$$n(K) = \lceil K \log_2(10) \rceil = 2601184$$

bits, represented exactly by

$$325148$$

bytes.

The total rounding gap of this complete block is approximately 5.20×10^{-7} bits, while the normalized per-symbol overhead is

$$\epsilon_{10}(783032) = \frac{5.20 \times 10^{-7}}{783032} \approx 6.64 \times 10^{-13}$$

bits per symbol.

In this context, representation proximity refers exclusively to the binary realization overhead and does not imply source compression.

Unlike statistical source coding methods, RICAN does not exploit source redundancy or probability distributions. The encoded length depends only on alphabet cardinality and input length.

The main contribution is a constructive procedure for selecting fixed-rate block representations with asymptotically vanishing absolute rounding gap through continued-fraction approximation.

A reference implementation using the RJ03 self-contained representation format is provided.

Index Terms

Fixed-rate representation, finite alphabets, continued fractions, Diophantine approximation, enumerative representation, bijective coding.

I. INTRODUCTION

Lossless information systems require reversible mappings between symbol sequences and binary descriptions.

Two different objectives are frequently associated with such mappings: source compression and binary representation.

Compression methods attempt to reduce expected description length by exploiting statistical properties of the source. In contrast, representation methods study the deterministic mapping between a finite alphabet and a binary storage space.

RICAN belongs to the second category.

The objective of RICAN is not to compress information content, but to minimize the unavoidable integer rounding overhead that appears when finite alphabet spaces are represented using binary hardware.

For an alphabet containing B symbols, the information quantity associated with one uniformly selected symbol is

$$\log_2(B)$$

bits.

When

$$B = 2^m,$$

the representation is exact because

$$\log_2(B) = m.$$

However, for alphabet cardinalities that are not powers of two, $\log_2(B)$ is generally irrational, preventing an exact one-symbol fixed-rate binary representation.

A block representation solves this problem by grouping symbols.

For a block length K , the number of possible sequences is

$$|\Sigma_B^K| = B^K.$$

A binary representation containing n bits provides

$$2^n$$

possible states.

Therefore, any reversible fixed-rate representation must satisfy

$$2^n \geq B^K.$$

Taking logarithms gives:

TABLE I
NOTATION SUMMARY

Symbol	Meaning
B	Alphabet cardinality
Σ_B	Alphabet of size B
K	Block length in symbols
$n(K)$	Binary representation length of a block
$G_B(K)$	Absolute representation gap
$\delta_B(K)$	Normalized representation overhead
$\epsilon_B(K)$	Per-symbol representation overhead
p_n/q_n	Continued-fraction convergent
L	Input sequence length
$N(L)$	Total representation length

$$n \geq K \log_2(B).$$

Because the number of bits must be an integer, the minimum required block length is:

$$n(K) = \lceil K \log_2(B) \rceil.$$

The resulting rounding overhead is:

$$G_B(K) = \lceil K \log_2(B) \rceil - K \log_2(B).$$

The central problem addressed by RICAN is the selection of block sizes K that minimize this deterministic gap.

The contribution of this work is a constructive method based on continued fractions.

For irrational values of $\log_2(B)$, the denominators of continued-fraction convergents provide block lengths where the product

$$K \log_2(B)$$

approaches an integer with increasing accuracy.

The framework is defined over the infinite family of finite alphabets:

$$\mathcal{A} = \{\Sigma_B : B \geq 2\}.$$

Therefore, binary alphabets, DNA alphabets, decimal digits, hexadecimal symbols, byte alphabets, and arbitrary finite alphabets are treated as particular instances of the same mathematical model.

The main contributions of this paper are:

- 1) A general fixed-rate binary representation mathematical framework for finite alphabets.
- 2) A constructive block-size selection procedure based on continued-fraction convergents of $\log_2(B)$.
- 3) A theoretical analysis proving the existence of convergent subsequences with vanishing absolute representation gap.
- 4) A self-contained reversible binary representation format (RJ03) that stores all information required for deterministic reconstruction.
- 5) An experimental validation over decimal sequences and multiple finite alphabets.

The remainder of this paper is organized as follows.

Section II reviews related approaches.

Section III introduces the mathematical foundations.

Section IV presents the main theoretical results.

Section V describes the block selection algorithm.

Section VI presents the RICAN representation framework and RJ03 format.

Section VII presents experimental evaluation.

Section VIII discusses implications and limitations.

Section IX presents code availability.

Section X concludes the paper.

II. NOTATION

The following notation is used throughout this paper.

The notation emphasizes the distinction between source size and representation size.

The alphabet cardinality B determines the ideal rate $\log_2(B)$, while the block length K determines the integer rounding behavior.

III. RELATED WORK

The problem of representing finite-alphabet information using binary hardware has been studied from several perspectives, including statistical coding, enumerative representation, and fixed-length mapping techniques.

RICAN differs from these approaches because its objective is not the reduction of expected source description length, but the minimization of deterministic binary representation overhead.

A. Statistical Source Coding

Classical source coding methods optimize representation length when statistical properties of the source are available.

Huffman coding assigns variable-length prefix codewords according to symbol probabilities and provides optimality among binary prefix codes for a known distribution [2].

Arithmetic coding represents sequences as subintervals of the unit interval and avoids the integer-length limitation of conventional prefix coding [4].

Asymmetric numeral systems (ANS) achieve entropy coding performance comparable to arithmetic coding while allowing efficient finite-state implementations [5].

These techniques exploit statistical redundancy.

Their objective is therefore different from RICAN, which constructs fixed-rate representations whose size depends only on alphabet cardinality and sequence length.

B. Dictionary-Based Methods

Dictionary-based methods reduce source description length by identifying repeated structures in sequences.

The Lempel-Ziv family introduced universal algorithms that represent repeated substrings through references to previously observed patterns [3].

Modern practical compression systems frequently combine dictionary methods with entropy coding to improve performance on structured data.

RICAN does not perform pattern detection, probability estimation, or redundancy removal.

Instead, it addresses the independent problem of representing every possible block of a finite alphabet using a deterministic binary mapping with minimum integer rounding overhead.

C. Enumerative Representation

Enumerative methods provide systematic ranking and unranking procedures for finite combinatorial spaces.

Cover introduced enumerative source encoding as a method for mapping sets of sequences into integer indices [6].

Later developments extended these ideas to constrained sequences and efficient combinatorial representations [7].

RICAN is related to enumerative representation because both approaches use the cardinality of finite spaces.

However, the primary objective differs.

Enumerative coding focuses on ranking specific classes of sequences, often under constraints.

RICAN focuses on the selection of block lengths K such that the binary representation length

$$\lceil K \log_2(B) \rceil$$

has minimum deterministic rounding error.

D. Fixed-Length Binary Representation

A finite alphabet with cardinality B requires at least

$$\lceil \log_2(B) \rceil$$

bits when symbols are represented independently.

This approach is optimal only when B is a power of two.

For general alphabets, block representation reduces the effect of rounding by considering the complete block space:

$$|\Sigma_B^K| = B^K.$$

The minimum binary length is therefore:

$$n(K) = \lceil K \log_2(B) \rceil.$$

RICAN studies the mathematical selection of K rather than the construction of a new coding probability model.

TABLE II
CONCEPTUAL COMPARISON BETWEEN REPRESENTATION AND COMPRESSION APPROACHES

Method	Objective	Statistics Required
Huffman Coding	Expected length minimization	Yes
Arithmetic Coding	Entropy-rate approximation	Yes
ANS	Entropy coding efficiency	Yes
Dictionary Methods	Pattern redundancy reduction	Yes
Enumerative Coding	Finite-space ranking	Sometimes
RICAN	Fixed-rate representation gap minimization	No

E. Continued Fractions and Diophantine Approximation

Continued fractions provide highly accurate rational approximations to irrational numbers. For an irrational number x , the continued fraction expansion is

$$x = [a_0; a_1, a_2, \dots].$$

The convergents are fractions

$$\frac{p_n}{q_n}$$

with the classical approximation property:

$$\left| x - \frac{p_n}{q_n} \right| < \frac{1}{q_n q_{n+1}}.$$

Multiplying by q_n gives:

$$|q_n x - p_n| < \frac{1}{q_{n+1}}.$$

Therefore, the denominators q_n identify integer multiples of x that approach integer values with increasing accuracy. RICAN applies this classical number-theoretic property to the selection of representation block sizes. For

$$x = \log_2(B),$$

the selected denominators provide block lengths where:

$$K \log_2(B)$$

is close to an integer boundary.

F. Position of RICAN

RICAN should be interpreted as a mathematical representation framework. Its contribution is not an entropy coding algorithm and not a compression mechanism. The framework provides:

- a general model for finite alphabet representation,
- a method for selecting fixed-rate block sizes,
- a theoretical analysis based on continued-fraction approximation,
- a self-contained reversible binary format.

The conceptual distinction between RICAN and conventional coding methods is summarized in Table II.

IV. PRELIMINARIES

This section introduces the mathematical definitions required for the RICAN framework.

A. Finite Alphabet Family

RICAN considers the family of finite alphabets:

$$\mathcal{A} = \{\Sigma_B : B \geq 2\},$$

where:

$$\Sigma_B = \{0, 1, \dots, B - 1\}.$$

The value B completely determines the representation problem.

Representative examples include:

- $B = 2$: binary alphabet.
- $B = 4$: DNA alphabet.
- $B = 10$: decimal digits.
- $B = 16$: hexadecimal alphabet.
- $B = 256$: byte alphabet.

B. Rationality of $\log_2(B)$

For an integer $B \geq 2$,

$$\log_2(B) \in \mathbb{Q}$$

if and only if

$$B = 2^m$$

for some integer $m \geq 1$.

If:

$$B = 2^m,$$

then:

$$\log_2(B) = m,$$

which is rational.

Conversely, assume:

$$\log_2(B) = \frac{p}{q}$$

in lowest terms.

Then:

$$B^q = 2^p.$$

By unique factorization, B can contain only the prime factor 2. Therefore:

$$B = 2^m$$

for some integer m .

C. Block Space

For a block length K , the set of possible input blocks is:

$$\Sigma_B^K.$$

Its cardinality is:

$$|\Sigma_B^K| = B^K.$$

A binary word of length n provides:

$$2^n$$

possible states.

Therefore, an injective representation requires:

$$2^n \geq B^K.$$

Taking logarithms:

$$n \geq K \log_2(B).$$

Because n must be an integer:

$$n(K) = \lceil K \log_2(B) \rceil.$$

D. Absolute Representation Gap

For an alphabet cardinality B , the absolute representation gap is:

$$G_B(K) = \lceil K \log_2(B) \rceil - K \log_2(B).$$

This quantity measures the integer rounding distance, in bits, between the ideal real-valued representation length and the actual binary block length.

E. Normalized Representation Overhead

The normalized representation overhead is:

$$\delta_B(K) = \frac{G_B(K)}{K \log_2(B)}.$$

The per-symbol representation overhead is:

$$\epsilon_B(K) = \frac{G_B(K)}{K}.$$

These quantities describe representation tightness relative to the ideal fixed-rate bound.

They do not represent compression efficiency.

F. Asymptotically Tight Representation

A sequence of block sizes K_i defines an asymptotically tight representation with respect to rounding overhead if:

$$\lim_{i \rightarrow \infty} \delta_B(K_i) = 0.$$

For RICAN, the selected sequence is obtained from suitable continued-fraction convergent denominators of $\log_2(B)$.

G. Fixed-Rate Representation Length

For an input sequence of length L , the total representation length is:

$$N(L) = \left\lceil \frac{L}{K} \right\rceil n(K),$$

where:

$$n(K) = \lceil K \log_2(B) \rceil.$$

This expression assumes the final incomplete block is encoded using its actual bit length and excludes constant container metadata.

For large K and large L , the effective rate approaches:

$$\log_2(B)$$

bits per symbol along the selected convergent subsequence.

H. Mathematical Relation to Enumerative Coding

RICAN and enumerative coding share the use of integer ranking over finite spaces.

In enumerative coding, sequences are mapped to integer indices through combinatorial counting.

In RICAN, the mapping is defined by radix conversion over complete blocks:

$$X(S) = \sum_{i=0}^{K-1} s_i B^{K-1-i}.$$

Both approaches produce reversible mappings.

However, the optimization criterion differs.

Enumerative coding typically minimizes expected index length under probability constraints.

RICAN minimizes deterministic rounding overhead at the block level through continued-fraction block selection.

Therefore, RICAN can be interpreted as a special case of enumerative representation where:

- the enumerated space is the complete Cartesian product Σ_B^K ,
- the objective is block-length optimization,
- the selection of K is performed by Diophantine approximation.

This connection clarifies the position of RICAN within existing representation techniques.

V. MAIN THEORETICAL RESULTS

This section establishes the mathematical foundation of RICAN.

The central observation is that the fixed-rate representation overhead depends on the distance between:

$$K \log_2(B)$$

and the nearest integer value.

For an alphabet cardinality B , define:

$$x_B = \log_2(B).$$

The representation gap is:

$$G_B(K) = \lceil K x_B \rceil - K x_B.$$

The objective of the RICAN construction is to select block lengths K for which this quantity becomes arbitrarily small.

A. Distance to Integer Values

For a real number y , define its distance to the closest integer as:

$$\text{dist}(y, \mathbb{Z}) = \min_{m \in \mathbb{Z}} |y - m|.$$

For:

$$x_B = \log_2(B),$$

define:

$$d_B(K) = \text{dist}(Kx_B, \mathbb{Z}).$$

This quantity measures how close the ideal representation length is to an integer number of bits. When the closest integer corresponds to the ceiling operation:

$$\lceil Kx_B \rceil,$$

the representation gap satisfies:

$$G_B(K) = d_B(K).$$

B. Continued-Fraction Approximation

Let x be irrational and let:

$$\frac{p_n}{q_n}$$

be a convergent of its continued-fraction expansion.

A classical property of continued fractions is:

$$\left| x - \frac{p_n}{q_n} \right| < \frac{1}{q_n q_{n+1}}.$$

Multiplying by q_n gives:

$$|q_n x - p_n| < \frac{1}{q_{n+1}}.$$

Because p_n is an integer:

$$\text{dist}(q_n x, \mathbb{Z}) \leq |q_n x - p_n|.$$

Therefore:

$$\text{dist}(q_n x, \mathbb{Z}) < \frac{1}{q_{n+1}}.$$

Let x be irrational and let

$$\frac{p_n}{q_n}$$

be a continued-fraction convergent of x .

Then:

$$\text{dist}(q_n x, \mathbb{Z}) < \frac{1}{q_{n+1}}.$$

The classical approximation theorem for continued fractions gives:

$$|q_n x - p_n| < \frac{1}{q_{n+1}}.$$

Since p_n belongs to $\mathbb{Z}$,

$$\text{dist}(q_n x, \mathbb{Z}) \leq |q_n x - p_n|.$$

Combining both expressions proves the result.

C. Convergent-Based Block Selection

The previous lemma directly provides a construction rule for RICAN.

For:

$$x_B = \log_2(B),$$

choose block sizes:

$$K = q_n,$$

where q_n are denominators of continued-fraction convergents.

Then:

$$Kx_B$$

approaches an integer value with an error bounded by:

$$\frac{1}{q_{n+1}}.$$

Let:

$$x_B = \log_2(B)$$

be irrational.

Let:

$$\frac{p_n}{q_n}$$

be the convergents of the continued-fraction expansion of x_B .

Then the block lengths:

$$K_n = q_n$$

satisfy:

$$\text{dist}(K_n x_B, \mathbb{Z}) < \frac{1}{q_{n+1}}.$$

Consequently, there exists an infinite sequence of RICAN block sizes whose ideal representation length approaches an integer value with arbitrarily small absolute error.

From Lemma V-B:

$$\text{dist}(q_n x_B, \mathbb{Z}) < \frac{1}{q_{n+1}}.$$

The denominators of continued-fraction convergents satisfy:

$$q_n \rightarrow \infty.$$

Therefore:

$$\frac{1}{q_{n+1}} \rightarrow 0.$$

Hence:

$$\lim_{n \rightarrow \infty} \text{dist}(q_n x_B, \mathbb{Z}) = 0.$$

This proves that the convergent denominators provide an infinite sequence of block sizes with vanishing distance between the ideal and integer representation lengths.

D. Existence of Vanishing Absolute Representation Gap

The previous theorem concerns distance to the nearest integer.

For RICAN, the relevant quantity is the ceiling gap:

$$G_B(K) = \lceil Kx_B \rceil - Kx_B.$$

Let:

$$x_B = \log_2(B)$$

be irrational.

Then there exists an infinite subsequence of continued-fraction convergent denominators:

$$K_i = q_{n_i}$$

such that:

$$G_B(K_i) \rightarrow 0.$$

Therefore, the absolute fixed-rate representation overhead of RICAN can be made arbitrarily small through convergent-based block selection.

From Theorem V-C, the convergent denominators satisfy:

$$\text{dist}(q_n x_B, \mathbb{Z}) < \frac{1}{q_{n+1}}.$$

The convergents alternate around the irrational number x_B . Therefore, infinitely many convergents approach x_B from above and infinitely many from below.

Consider the subsequence approaching from above:

$$\frac{p_{n_i}}{q_{n_i}} > x_B.$$

For sufficiently large indices:

$$p_{n_i} = \lceil q_{n_i} x_B \rceil.$$

Therefore:

$$G_B(q_{n_i}) = p_{n_i} - q_{n_i} x_B.$$

Using the continued-fraction bound:

$$G_B(q_{n_i}) < \frac{1}{q_{n_i+1}}.$$

Since:

$$q_{n_i+1} \rightarrow \infty,$$

we obtain:

$$\lim_{i \rightarrow \infty} G_B(q_{n_i}) = 0.$$

Therefore, the selected convergent subsequence provides block sizes with vanishing absolute representation gap.

The result is specific to the selected convergent subsequence.

The normalized overhead:

$$\frac{G_B(K)}{K}$$

may decrease for many increasing block-size sequences.

However, arbitrary growth of K does not guarantee that the absolute integer rounding gap decreases.

RICAN provides a deterministic selection rule through continued fractions that identifies block lengths with exceptionally small absolute rounding error.

Algorithm 1 RICAN Block Selection Using Continued Fractions

Require: Alphabet cardinality $B \geq 2$, target gap ϵ

Ensure: Block size K , binary length $n(K)$

1: Compute:

$$x \leftarrow \log_2(B)$$

2: Generate continued-fraction expansion of x

3: Compute convergents:

$$p_i/q_i$$

4: **for** each convergent **do**

5:

$$K \leftarrow q_i$$

6:

$$n(K) \leftarrow \lceil Kx \rceil$$

7:

$$G \leftarrow n(K) - Kx$$

8: **if** $G < \epsilon$ **then**

9: **return** $(K, n(K), G)$

10: **end if**

11: **end for**

VI. BLOCK SIZE SELECTION ALGORITHM

The theoretical results provide a constructive procedure for selecting RICAN block sizes.

Given an alphabet cardinality B and a desired representation gap threshold ϵ , the algorithm computes continued-fraction convergents of:

$$\log_2(B).$$

Each denominator is evaluated as a candidate block size.

For each candidate:

$$K = q_i,$$

the required binary length is computed:

$$n(K) = \lceil K \log_2(B) \rceil.$$

The representation gap is:

$$G_B(K) = n(K) - K \log_2(B).$$

For the decimal alphabet:

$$B = 10,$$

the reference implementation uses:

$$K = 643.$$

This block size provides a practical balance between arithmetic complexity and representation tightness.

Larger convergent-derived values, such as:

$$K = 783032,$$

demonstrate the asymptotic behavior of the construction but require larger integer arithmetic operations.

All numerical evaluations of convergents should use arbitrary precision arithmetic to avoid floating-point rounding errors.

VII. REPRESENTATION FRAMEWORK

A. General Encoding Model

The RICAN encoding process constructs an injective mapping from finite alphabet blocks into binary words. Given an input sequence:

$$S = s_0 s_1 \dots s_{L-1}$$

over an alphabet:

$$\Sigma_B$$

with cardinality B , and a selected block size K , the sequence is partitioned into blocks. For a complete block:

$$B_i = (s_0, s_1, \dots, s_{K-1}),$$

the corresponding integer value is:

$$X(B_i) = \sum_{j=0}^{K-1} s_j B^{K-1-j}.$$

The possible values satisfy:

$$0 \leq X(B_i) < B^K.$$

Since:

$$B^K \leq 2^{n(K)},$$

where:

$$n(K) = \lceil K \log_2(B) \rceil,$$

each block can be represented uniquely using exactly $n(K)$ bits.

The final incomplete block, when present, is encoded using its actual length:

$$K' < K.$$

The decoder reconstructs the original sequence by applying the inverse integer-to-symbol mapping using repeated division by B .

B. Injective Representation Mapping

For a fixed alphabet and block size, the encoder is defined as:

$$E_B : \Sigma_B^L \rightarrow \{0, 1\}^{N(L)}.$$

The mapping is injective because each valid input block is assigned a unique integer value. The decoder is the inverse mapping over the image of the encoder:

$$D_B : \text{Im}(E_B) \rightarrow \Sigma_B^L.$$

Therefore:

$$D_B(E_B(S)) = S$$

for every valid input sequence S .

The binary representation space may contain unused states because:

$$2^{n(K)} > B^K$$

for non-power-of-two alphabets.

These unused states do not affect reversibility because the decoder operates only over valid encoded representations.

TABLE III
RJ03 SELF-CONTAINED REPRESENTATION FORMAT

Offset	Size	Field
0	4	Magic identifier “RJ03”
4	4	Format version
8	4	Alphabet cardinality B
12	8	Block size K
20	8	Bits per block $n(K)$
28	4	Alphabet length
32	variable	Alphabet symbols
–	8	Original sequence length L
–	8	Complete block count
–	8	Residual block size
–	variable	Encoded bitstream

C. Power-of-Two Alphabets

For:

$$B = 2^m,$$

the representation is exact:

$$n(K) = \lceil Km \rceil = Km.$$

Therefore:

$$G_B(K) = 0.$$

Examples include:

- Binary alphabet ($B = 2$): one bit per symbol.
- DNA alphabet ($B = 4$): two bits per symbol.
- Hexadecimal alphabet ($B = 16$): four bits per symbol.
- Byte alphabet ($B = 256$): eight bits per symbol.

D. The RJ03 Self-Contained Representation Format

The reference implementation uses the RJ03 binary representation format.

RJ03 is designed as a self-contained reversible representation container.

Unlike entropy-coded formats, RJ03 does not require:

- probability models,
- adaptive states,
- external dictionaries,
- training data.

All parameters required for reconstruction are stored inside the representation file.

The format contains:

- alphabet definition,
- alphabet cardinality,
- block size,
- original sequence length,
- encoded block stream.

The binary fields use unsigned little-endian representation.

Unused bits in the final byte are padding bits and are ignored during decoding.

VIII. EXPERIMENTAL EVALUATION

The experiments evaluate the mathematical representation properties of RICAN.

The objective is not compression benchmarking.

The experiments verify:

- 1) exact reconstruction,
- 2) agreement between theoretical and practical representation sizes,
- 3) independence of representation length from source statistics,
- 4) applicability to multiple finite alphabets.

TABLE IV
DECIMAL REPRESENTATION VERIFICATION

Parameter	Value
Input symbols	1,000,508
Alphabet size	10
Block size	643
Bits per block	2136
Encoded size	415,474 bytes
Verification	PASS

TABLE V
REPRESENTATION-LENGTH INDEPENDENCE

Source Type	Encoded Size
Mathematical constant digits	Same
Random decimal sequence	Same
Structured decimal sequence	Same

A. Decimal Representation Verification

A decimal sequence containing:

$$L = 1\,000\,508$$

digits was encoded with:

$$B = 10$$

and:

$$K = 643.$$

The theoretical block size is:

$$n(K) = 2136$$

bits.

The measured container size is shown in Table IV.

The measured size includes the RJ03 container overhead required for self-contained reconstruction.

B. Representation-Length Independence

Because RICAN is a fixed-rate representation framework, the output size depends on:

$$B$$

and:

$$L.$$

It does not depend on symbol probability distribution.

Sequences with different statistical properties but identical length and alphabet cardinality produce identical representation sizes.

C. Alphabet Generalization

The same framework applies to multiple finite alphabets.

IX. DISCUSSION

A. Main Contribution

The main contribution of RICAN is the construction of fixed-rate representation schemes whose integer rounding overhead can be reduced through continued-fraction block selection.

The existence of fixed-length representations follows directly from finite-space cardinality.

RICAN contributes a deterministic method for selecting block sizes with exceptionally small representation gaps.

TABLE VI
FINITE ALPHABET VERIFICATION

Alphabet	B	Result
Binary	2	PASS
DNA	4	PASS
Decimal	10	PASS
Hexadecimal	16	PASS
ASCII	128	PASS

B. Relationship With Compression

RICAN does not reduce source information.

It does not exploit:

- symbol frequencies,
- repeated patterns,
- statistical dependencies.

Therefore, RICAN should not be considered a replacement for entropy coding methods.

Its objective is the optimization of deterministic binary representation overhead.

C. Limitations

The current framework has several limitations.

Very large convergent block sizes require arbitrary precision integer arithmetic.

The representation format introduces constant metadata overhead.

Additionally, because the method does not use probability models, it does not provide compression gains for redundant sources.

X. CODE AVAILABILITY AND REPRODUCIBILITY

The reference implementation, RJ03 specification, and evaluation scripts are publicly available at:

<https://github.com/SequenceOfManyZerosAndOnes/RICAN>

The repository contains:

- encoder and decoder implementation,
- RJ03 format documentation,
- reproducible examples,
- mathematical documentation.

The experiments were performed using:

Python 3.13

RICAN RJ03 reference implementation

A permanent archival release through Zenodo is planned.

Software version:

RICAN RJ03 v1.0.0 – July 2026

XI. CONCLUSION

This paper presented RICAN, a fixed-rate binary representation framework for finite alphabets based on continued-fraction convergent selection.

The framework applies uniformly to:

$$\mathcal{A} = \{\Sigma_B : B \geq 2\}.$$

For power-of-two alphabets:

$$B = 2^m,$$

the representation is exact.

For non-power-of-two alphabets, continued-fraction convergents of $\log_2(B)$ provide block lengths whose absolute representation gap approaches zero along the selected subsequence.

For decimal representation:

$$B = 10,$$

the practical implementation uses:

$$K = 643,$$

while larger convergent-derived values demonstrate the asymptotic behavior of the construction.

RICAN provides a mathematical framework for studying the boundary between finite-alphabet representation and binary storage.

Future research directions include:

- large-scale DNA representation,
- hardware acceleration,
- parallel implementations,
- optimization of very large convergent blocks.

REFERENCES

- [1] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 623–656, 1948.
- [2] D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.
- [3] J. Ziv and A. Lempel, "A Universal Algorithm for Sequential Data Compression," *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, 1977.
- [4] J. Rissanen and G. G. Langdon, "Arithmetic Coding," *IBM Journal of Research and Development*, vol. 23, no. 2, pp. 149–162, 1979.
- [5] J. Duda, "Asymmetric numeral systems: Entropy coding combining speed of Huffman coding with compression ratio of arithmetic coding," arXiv:1311.2540, 2013.
- [6] T. M. Cover, "Enumerative Source Encoding," *IEEE Transactions on Information Theory*, vol. 19, no. 1, pp. 73–77, 1973.
- [7] K. A. S. Immink, "A Survey of Enumerative Coding Techniques," *IEEE Transactions on Information Theory*, vol. 50, no. 10, pp. 2276–2285, 2004.
- [8] G. H. Hardy and E. M. Wright, *An Introduction to the Theory of Numbers*, 6th ed., Oxford University Press, 2008.
- [9] A. Ya. Khinchin, *Continued Fractions*, Dover Publications, 1997.
- [10] J. W. S. Cassels, *An Introduction to Diophantine Approximation*, Cambridge University Press, 1957.
- [11] A. M. Rockett and P. Szűsz, *Continued Fractions*, World Scientific, 1992.
- [12] R. P. Brent and P. Zimmermann, *Modern Computer Arithmetic*, Cambridge University Press, 2010.